

Desenvolvimento Web 2

Aula 1

Revisão da Linguagem JavaScript

`l.bertholdo@ifsp.edu.br`

Conteúdo

- Histórico
- Características
- Tipos de Dados, Variáveis e Escopo de Variáveis
- Eventos
- Funções
- Estruturas Condicionais e de Repetição
- Document Object Model (DOM)
- Manipulação de Strings e Datas
- Expressões Regulares

Histórico



- Em dezembro de 1995, a Sun Microsystems (atual Oracle), em parceria com a Netscape, anunciou o lançamento da linguagem **JavaScript**, a qual foi criada por **Brendan Eich** da Netscape.
- Em março de 1996, a Netscape lançou o navegador Netscape 2.0 com suporte para a recém-criada linguagem.
- Em outubro de 1996, as especificações da linguagem foram submetidas à apreciação da **ECMA International (European Computer Manufacturers Association)**, associação fundada em 1961 dedicada à padronização de sistemas de informação.
- Em junho de 1997, foi publicada a **ECMAScript 1**, primeira versão padronizada da JavaScript.
- A partir disso, o nome oficial da linguagem passou a ser **ECMAScript**, sendo a versão mais recente a **ECMAScript 16**, publicada em junho de 2025.

Histórico

- Versões anteriores
 - ECMA-262, 1st edition, June 1997
 - ECMA-262, 2nd edition, August 1998
 - ECMA-262, 3rd edition, December 1999
 - ECMA-262, 4th edition (not existing)
 - **ECMA-262, 5th edition, December 2009**
 - ECMA-262, 5.1th edition, June 2011
 - ECMA-262, 6th edition, June 2015
 - ECMA-262, 7th edition, June 2016
 - ECMA-262, 8th edition, June 2017
 - ECMA-262, 9th edition, June 2018
 - ECMA-262, 10th edition, June 2019
 - ECMA-262, 11th edition, June 2020
 - ECMA-262, 12th edition, June 2021
 - ECMA-262, 13th edition, June 2022
 - ECMA-262, 14th edition, June 2023
 - ECMA-262, 15th edition, June 2024

*Embora tenham sido lançadas versões mais recentes da ECMAScript, atualmente, a versão mais bem suportada pelos navegadores ainda é a **ECMAScript 5**, criada em 2009.*

*Para verificar o suporte à **ECMAScript 6** nos principais navegadores, acesse <https://caniuse.com/?search=es6>.*

Características

- Transpilers
 - Os navegadores atuais não dão total suporte à **ECMAScript 6** e versões posteriores, mas é possível usá-las graças aos ***transpilers*** – compiladores capazes de traduzir o código escrito em uma versão de uma linguagem para código equivalente em outra versão da mesma linguagem.
 - Atualmente, o *transpiler* JavaScript mais conhecido e usado é o **Babel**, que converte o código **ECMAScript 6** para **ECMAScript 5**.
 - Assim, as sintaxes do código **ECMAScript 6**, submetidas ao *transpiler*, são entendidas pelos navegadores.

Para verificar o suporte dos atuais transpilers à **ECMAScript 6**, acesse <https://compat-table.github.io/compat-table/es6/>.

*Além da **ECMAScript 6**, **TypeScript** e **JSX (JavaScript Syntax eXtension)** são exemplos de sintaxes que necessitam de transpilers para serem suportadas pelos navegadores.*

Características

- **JavaScript** é uma linguagem de programação interpretada, com tipagem dinâmica e fraca, utilizada principalmente no desenvolvimento *cliente-side* de aplicações Web.

*Em linguagens com tipagem **dinâmica** e **fraca**, o tipo de dado de uma variável não precisa ser informado, pois é determinado (ou modificado) em tempo de execução. Além disso, são linguagens que permitem conversões implícitas (automáticas) de tipos. Por exemplo:*

```
console.log(10 + "10"); // 1010
```

- Enquanto HTML e CSS tem como finalidades básicas, respectivamente, criar uma **estrutura** e tratar a **estética** de páginas Web, **JavaScript** é usada para adicionar **capacidade de processamento, geração dinâmica de conteúdo e interatividade avançada** às páginas.

Tipos de Dados

- JavaScript possui tipagem dinâmica, ou seja, não é necessário definir o tipo de dado das variáveis ao declará-las. No entanto, a linguagem permite trabalhar com seis tipos de dados literais:

Tipos de Dados	Descrição
Inteiro	Números em base 10 (positivos e negativos) e em base 16 (hexadecimais). Números hexadecimais devem ser precedidos de "0x". Exemplo: 0xFF00FF.
Decimal	Números fracionários com casas decimais separadas por ponto. Exemplo: 3.1416.
Booleano	Palavras-chave <code>true</code> e <code>false</code> (em letras minúsculas).
Cadeia de caracteres	Conjunto de zero ou mais caracteres envolvidos por aspas duplas ou simples. Uma string também pode conter sequências de escape, como <code>"\n"</code> para pular uma linha. Exemplos: "Maria da Silva", 'Carlos Alberto'.
Array	Conjuntos de zero ou mais valores, de diferentes tipos ou não, separados por vírgula e envolvidos por colchetes. Cada valor contido em um <i>array</i> possui um índice sequencial iniciado em zero. Exemplos: cores = ["azul", "verde", "preto"]; dados = ["João", 35, 1.78, true];
Objeto	Conjuntos de pares nome/valor separados por vírgulas e envolvidos por chaves. Cada par pode ser: uma <u>propriedade</u> do objeto e seu respectivo valor, um <u>método do objeto</u> (que retorna um determinado valor), ou ainda um <u>outro objeto</u> . Exemplo: Carro = { <code>marca</code> : "Ford", <code>modelo</code> : "Ka", <code>ano</code> : 2014, <code>ipva</code> : "calcula()", <code>dimensao</code> : { <code>comprimento</code> : 3.9, <code>largura</code> : 1.5 } };

Variáveis

- Até a **ECMAScript 5**, a única palavra reservada disponível para declarar variáveis era a palavra **var**.
- Uma variável é **declarada** quando atribuímos um nome a ela, e é **inicializada** quando atribuímos um valor ao nome. Essas duas ações podem ser escritas separadamente ou de uma só vez:

```
var peso, altura; // Variáveis declaradas.  
peso = 65.5; // Variável inicializada (declarada na linha de cima).  
var nome = "João", idade = 28; // Variáveis declaradas e inicializadas.
```

- Como JavaScript tem uma tipagem dinâmica, uma mesma variável pode armazenar qualquer tipo de dado em um mesmo script:

```
var idade = 32;  
idade = "trinta e dois";
```

A linguagem reconhece o tipo de dado quando um valor é atribuído à variável.

Escopo de Variáveis

- Escopo de uma variável é a região do script onde a variável é conhecida, ou seja, onde seu valor pode ser acessado. As especificações anteriores à **ECMAScript 6** definem dois escopos para as variáveis: **global** e **local**.
- Uma **variável global** pode ser acessada em todas as regiões do script, exceto se ela for redeclarada dentro de uma função, pois a variável torna-se **local**. Uma **variável local** pode ser acessada apenas dentro da função onde foi declarada.

*JavaScript possui um mecanismo chamado **hoisting**. Ele permite que uma variável possa ser usada (como na linha 4) antes mesmo de ser declarada explicitamente.*

Função chamada ao clicar em um botão.

```
2 var x = 1; // Variável global
3 function verificaVariavel() {
4     console.log(x) // Retorna "undefined"
5     var x = 5;     // Variável Local
6     console.log(x) // Retorna 5
7 }
8 console.log(x)    // Retorna 1
```

Ao redeclarar a variável **x** na linha 5, ela é “elevada” para o topo do contexto da função, ou seja, é como se a 1ª linha da função passasse a ter a declaração **var x**;
Ao acessá-la na linha 4, como ela foi declarada sem ser inicializada, o valor atribuído a ela é “**undefined**”.

1	scripts3.1.js:8
undefined	scripts3.1.js:4
5	scripts3.1.js:6

Escopo de Variáveis

- Até a **ECMAScript 5**, a única região de **escopo local** que podia ser criada era aquela relacionada às **funções** e por meio da palavra **var**.
- Para resolver problemas relacionados ao escopo das variáveis em códigos mais extensos, a **ECMAScript 6** criou as palavras reservadas **let** e **const**, que definem novas regiões de **escopo local** chamadas **escopo de bloco**.
- Variáveis **let** podem ser **atualizadas**, mas, diferentemente das variáveis **var**, não podem ser **redeclaradas** dentro do seu escopo. Além disso, o escopo de uma variável **let** limita-se ao local onde ela foi declarada.

```
2 function verificaVariavel() {
3   if (true) {
4     var escopoFuncao = 'João';
5     let escopoBloco = 'Maria';
6     console.log(escopoBloco); // Retorna "Maria"
7     // let escopoBloco = 'Pedro'; // Retornaria "Uncaught SyntaxError"
8   }
9   console.log(escopoFuncao); // Retorna "João"
10  // console.log(escopoBloco); // Retornaria "Uncaught ReferenceError"
11 }
```

Como foi definida com **let**, a variável **escopoBloco** pode atualizada, mas não pode ser redeclarada dentro do seu escopo. Além disso, não pode ser acessada fora do **if**.

Maria	scripts3.2.js:6
João	scripts3.2.js:9

Escopo de Variáveis

- Variáveis **const** não podem ser **atualizadas** nem **redeclaradas** dentro do seu escopo. E assim como ocorre com variáveis **let**, o escopo de uma variável **const** também limita-se ao local onde ela foi declarada.
- Diferentemente das variáveis **var** e **let**, variáveis **const** devem ser obrigatoriamente inicializadas no momento da declaração.

```
2 function verificaVariavel() {  
3   if (true) {  
4     var escopoFuncao = 'João';  
5     const escopoBloco = 'Maria';  
6     console.log(escopoBloco);           // Retorna "Maria"  
7     // escopoBloco = 'Pedro';           // Retornaria "Uncaught TypeError"  
8     // const escopoBloco = 'Pedro';     // Retornaria "Uncaught SyntaxError"  
9   }  
10  console.log(escopoFuncao);             // Retorna "João"  
11  // console.log(escopoBloco);           // Retornaria "Uncaught ReferenceError"  
12 }
```

Como foi definida com **const**, a variável **escopoBloco** não pode ser atualizada e nem redeclarada dentro do seu escopo. Além disso, não pode ser acessada fora do **if**.

Maria	scripts3.3.js:6
João	scripts3.3.js:10

*Ao declarar uma variável sem as palavras **var**, **let** ou **const**, ela é criada no escopo global, mesmo que tenha sido declarada dentro de uma função. É uma boa prática sempre usar uma das palavras **var**, **let** ou **const**, evitando sempre que possível a utilização de variáveis globais por meio da palavra **var**.*

Eventos

- Geralmente, a execução dos *scripts* é disparada a partir de ações ou ocorrências comuns que acontecem em um sistema durante a sua operação. Estas ações ou ocorrências são chamadas de **eventos**.
- Alguns exemplos de eventos são: o clique em um botão, o carregamento de uma página, o envio de um formulário etc.
- Para associar os *scripts* a esses tipos de ocorrências, a linguagem HTML disponibiliza vários atributos para captura de eventos em seus elementos.

Eventos

- Alguns atributos HTML para captura de eventos:

Tipos de Evento	Atributos	Descrição
Eventos de janela	onload	Disparado quando o navegador termina de carregar a página.
	onunload	Disparado quando o navegador é fechado.
	onresize	Disparado quando a janela do navegador é redimensionada.
Eventos de formulário	onfocus	Disparado quando um campo de formulário recebe o foco.
	onblur	Disparado quando um campo de formulário perde o foco.
	onchange	Disparado quando o valor de um campo é alterado.
	onsubmit	Disparado quando o formulário é enviado.
Evento de teclado	onkeypress	Disparado quando uma tecla é pressionada e solta sobre um elemento selecionado.
Eventos de mouse	onclick	Disparado quando se clica sobre um elemento.
	ondblclick	Disparado quando se dá um duplo clique sobre um elemento.
	onmouseover	Disparado quando se passa o cursor do mouse sobre um elemento.

Funções

- Bloco de instruções capaz de realizar tarefas que retornam um determinado resultado.
- As funções tornam o código mais organizado, pois as microfuncionalidades da aplicação ficam separadas umas das outras, facilitando assim a **manutenção**.
- Além disso, as funções permitem o **reuso de código**, já que podem ser chamadas sempre que necessário.

***Exemplo:** Uma função para validar se um campo foi preenchido somente com letras pode ser chamada em todos os campos que aceitam exclusivamente letras.*

Funções

```
<input type="button" value="Verificar % de Frequência" onclick="verificaFrequencia()">
```

```
function verificaFrequencia() {  
    var frequencia = (document.frequencia.faltas.value / 15) * 100;  
  
    if (frequencia < 75)  
        alert("O aluno tem " + frequencia + " % de presença\n e está sujeito à reprovação por faltas.");  
    else  
        alert("O aluno tem " + frequencia + " % de presença\n e está com a frequência em dia.");  
}
```

Verificação de frequência dos alunos

RA

Nome

Faltas

Essa página diz

O aluno tem 60 % de presença
e está sujeito à reprovação por faltas.

Estruturas Condicionais

- Permitem avaliar uma expressão e, conforme o resultado obtido, executar um determinado trecho de código.

```
<button type="submit" onclick="alert(verificaClasse(15000))">Verificar Classe</button>  
<button type="submit" onclick="alert(verificaCor('azul'))">Verificar Classe</button>
```

```
function verificaClasse(renda) {  
  if (renda >= 20000)  
    return "Classe A";  
  else if (renda >= 10000)  
    return "Classe B";  
  else  
    return "Classe C";  
}
```

```
function verificaCor(cor) {  
  switch (cor) {  
    case "amarelo":  
      return "A cor é amarelo.";  
      break;  
    case "verde":  
      return "A cor é verde.";  
      break;  
    case "azul":  
      return "A cor é azul.";  
      break;  
    default:  
      return "Cor inválida!";  
  }  
}
```


Estruturas de Repetição

- Permitem executar um trecho de código por um determinado número de vezes, ou até que uma condição seja satisfeita.

```
<button type="submit" onclick="alert('Existe(m) ' + verificaIdadeComWhile(65) +  
    ' pessoa(s) com 65 anos ou mais.')">Verificar Idades (com while)</button>  
<button type="submit" onclick="alert('Existe(m) ' + verificaIdadeComFor(65) +  
    ' pessoa(s) com 65 anos ou mais.')">Verificar Idades (com for)</button>
```

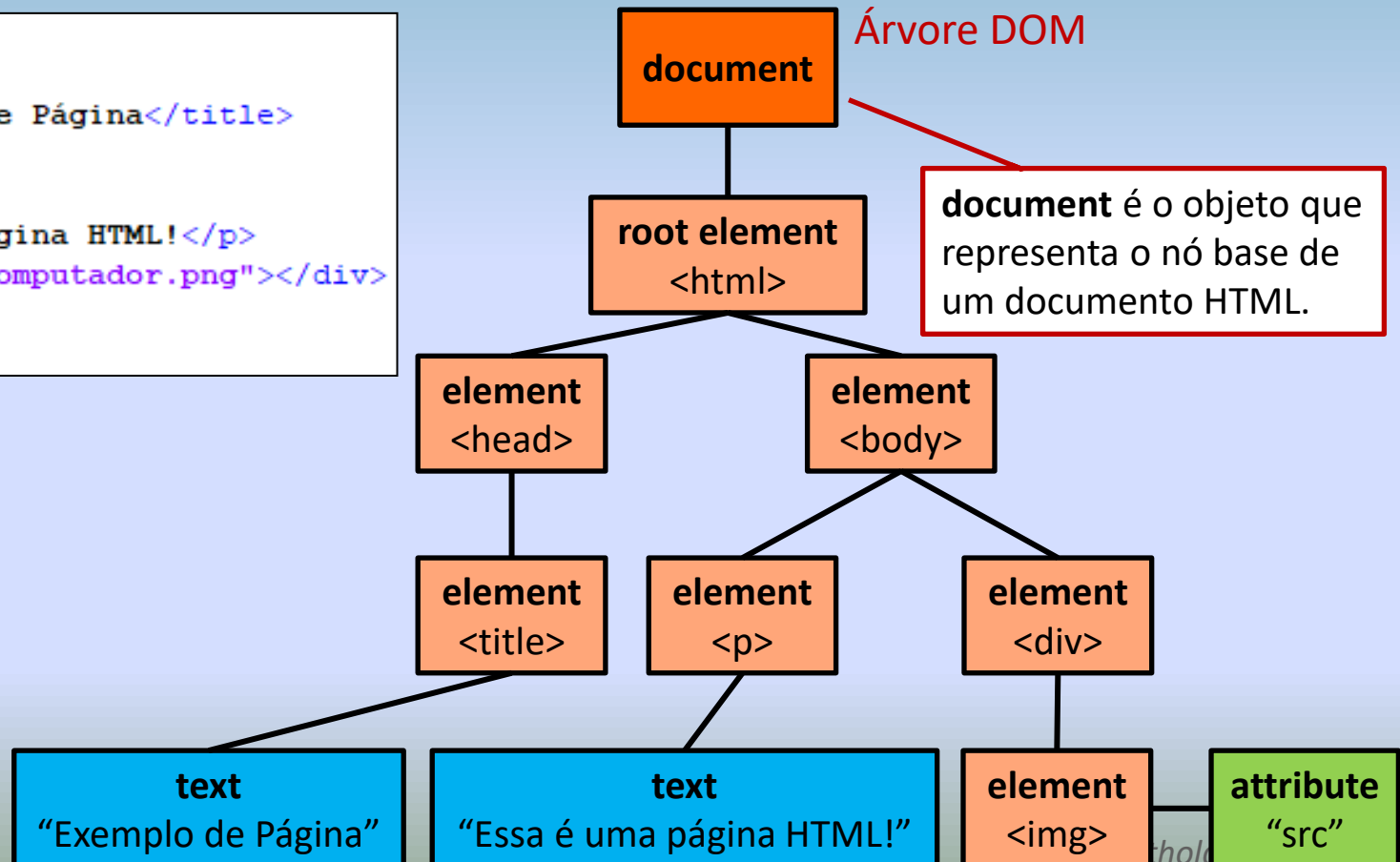
```
function verificaIdadeComWhile(idadeMinima) {  
    var idades = [15, 28, 11, 67, 75];  
    var i = 0, cont = 0;  
    while (i < 5) {  
        if (idades[i] >= idadeMinima)  
            cont++;  
        i++;  
    };  
    return cont;  
}
```

```
function verificaIdadeComFor(idadeMinima) {  
    var idades = [15, 28, 11, 67, 75];  
    var cont = 0;  
    for (var i = 0; i < 5; i++) {  
        if (idades[i] >= idadeMinima)  
            cont++;  
    };  
    return cont;  
}
```

Document Object Model (DOM)

- O modelo DOM organiza cada elemento de um documento HTML em uma estrutura de árvore, permitindo às aplicações acessá-los de forma padronizada.

```
<html>
<head>
  <title>Exemplo de Página</title>
</head>
<body>
  <p>Essa é uma página HTML!</p>
  <div></div>
</body>
</html>
```



Document Object Model (DOM)

- Objeto **document**

- Toda vez que se abre um documento HTML no navegador é criado um objeto **document**.
- Com ele é possível referenciar, via JavaScript, qualquer elemento HTML de uma página.

```
<form name="clientes" method="post">  
  <!-- Outros componentes de interface -->  
  <input type="text" name="nome" value="João da Silva">  
  <!-- Outros componentes de interface -->  
</form>
```

`document.clientes.nome.value`

Referenciando o valor
do elemento **<input>**
por meio de JavaScript.

Atributo "name"
do formulário

Atributo "name" do
campo de texto "Nome"

Atributo "value" do campo
de texto "Nome"

Document Object Model (DOM)

- Métodos para acessar elementos pelos atributos **id** e **name**
 - **getElementById**: Permite referenciar um elemento pelo valor que foi definido em seu atributo **id**.

```
<input type="text" id="resultado">
```

```
document.getElementById("resultado").value = 1578;
```

- **getElementsByName**: Permite referenciar todos os elementos que tem o mesmo valor definido no atributo **name**.

```
<input type="radio" name="estados" value="SP">  
<input type="radio" name="estados" value="RJ">  
<input type="radio" name="estados" value="MG">
```

```
var result = document.getElementsByName("estados");  
var estado1 = result[0].value;  
var estado2 = result[1].value;  
var estado3 = result[2].value;
```

Document Object Model (DOM)

- Definição de estilos com JavaScript
 - **style:** Permite aplicar propriedades CSS a um elemento HTML.

```
<input type="text" name="nome" id="nomeId">
```

```
document.clientes.nome.style.backgroundColor = "blue";  
ou  
document.getElementById("nomeId").style.backgroundColor = "blue";
```

- Em JavaScript, as propriedades CSS compostas de duas palavras são escritas sem traço e no formato **camelCase**. Exemplos:

CSS	JavaScript
background-color	backgroundColor
text-align	textAlign
font-size	fontSize

Document Object Model (DOM)

- Propriedade **innerHTML**
 - Permite verificar ou definir o conteúdo de elementos HTML que possuem *tags* de abertura e fechamento, como: `<p>`, `<h1>`, `<div>`, `<nav>`, `<section>` etc.

```
document.getElementById("media").innerHTML = "Média: 8.2";
```

```
<p id="media">Média:</p>
```

Ao executar o código JavaScript, o conteúdo da *tag* `<p>` é alterado de "Média:" para "Média: 8.2".

Manipulação de Strings

- Em JavaScript, uma string pode ser manipulada por meio de propriedades e métodos.
- Propriedade **length**: Retorna o número de caracteres de uma string.
- Método **replace**: Permite substituir trechos de strings existentes dentro de outras strings.
- Método **substring**: Permite criar uma nova string a partir de um trecho de outra string.

```
var texto = "Pindamonhangaba";  
alert("Pindamonhangaba tem " + texto.length + " caracteres.");
```

Pindamonhangaba tem 15 caracteres.

```
var bairro = "Jardim Brasil";  
alert(bairro.replace("Jardim", "Jd."));
```

Jd. Brasil

```
var horario = "15:27:33";  
alert(horario.substring(0,2) + " horas, " +  
      horario.substring(3,5) + " minutos e " +  
      horario.substring(6) + " segundos");
```

15 horas, 27 minutos e 33 segundos

Manipulação de Datas

- A linguagem JavaScript possui um objeto chamado **Date**, destinado à manipulação de datas e horários.
 - Método **getTime**: Retorna a quantidade de milissegundos transcorridos entre a data definida em um objeto **Date** e a data base de 01/01/1970.

```
// Equivale a 20/06/2021 às 08:25:47 (os meses vão de 0 à 11).  
// Também é possível informar apenas ano, mês e dia (sem o horário).  
var data = new Date(2021, 5, 20, 8, 25, 47);  
alert(data.getTime());
```

1624188347000

O valor em milissegundos pode ser usado para realizar operações com datas e horários. Por exemplo, verificar quantos dias têm entre uma data e outra.

Expressões Regulares

- A linguagem JavaScript possui um objeto chamado **RegExp** que permite criar expressões regulares, as quais podem ser usadas para buscar padrões de caracteres.
- Os padrões a serem buscados devem ser colocados entre barras e separados pelo caractere “|”. Por exemplo: `/noite|Noite/`.
- Outros exemplos:
 - `[0-9]` – A busca é feita para os números de 0 a 9.
 - `[^0-9]` – A busca é feita para os caracteres que não sejam os números de 0 a 9.
 - `[a-z]` – A busca é feita para as letras de a até z (minúsculas).
 - `[A-Z]` – A busca é feita para as letras de A até Z (maiúsculas).
 - `[a-zA-Z]` – A busca é feita para as letras a até z (maiúsculas e minúsculas).
 - `[a|e|i|o|u]` – A busca é feita para os caracteres a, e, i, o ou u.

Expressões Regulares

- O método **test()** do objeto `RegExp` permite verificar se uma *string* está de acordo com o padrão definido por uma expressão regular.
- **Exemplo:** Supondo que um fórum deseja impedir que existam discussões sobre política, futebol e religião entre os usuários. Ao postar uma nova mensagem, uma função poderia verificar se ela possui palavras relacionadas a estes assuntos e, se possuir, emitir uma mensagem para o usuário.

```
var assuntosIndevidos = /futebol|política|religião/;

if (assuntosIndevidos.test(document.forum.mensagem.value))
    alert("Este fórum não permite discussões sobre futebol, política" +
        "e religião. Por este motivo, sua mensagem não será enviada.");
```

Referências

- SILVA, Maurício S. **JavaScript Essencial**. São Paulo: Editora Novatec, 2024.
- SILVA, Maurício S. **React: Aprenda Praticando**. São Paulo: Editora Novatec, 2021.